**Dipl.-Ing. Michael Zimmermann**
Buchenstr. 15
42699 Solingen
☎ 0212 46267
https://kruemelsoft.hier-im-netz.de
BwMichelstadt@t-online.de

Michelstadt (Bw)




Der in diesem Dokument beschriebene Workflow hat dem Autor geholfen, seinen Raspberry Pi als PiLocoBuffer einzurichten.

Der Autor dieser Anleitung übernimmt keine Garantie oder Haftung für Schäden oder Mängel, die sich durch die Verwendung dieser Anleitung ergeben.

Für Hinweise auf Fehler oder Ergänzungen ist der Autor dankbar.

Die Nennung von Marken- und Firmennamen geschieht in rein privater und nichtgewerblicher Nutzung und ohne Rücksicht auf bestehende Schutzrechte.

---

*Diese Zusammenstellung wurde nach bestem Wissen*
*und ohne Funktionsgarantie in der Hoffnung erstellt, dass sie nützlich ist.*
*Wenn sie nicht nützlich ist – dann eben nicht.*

---

# Übersicht

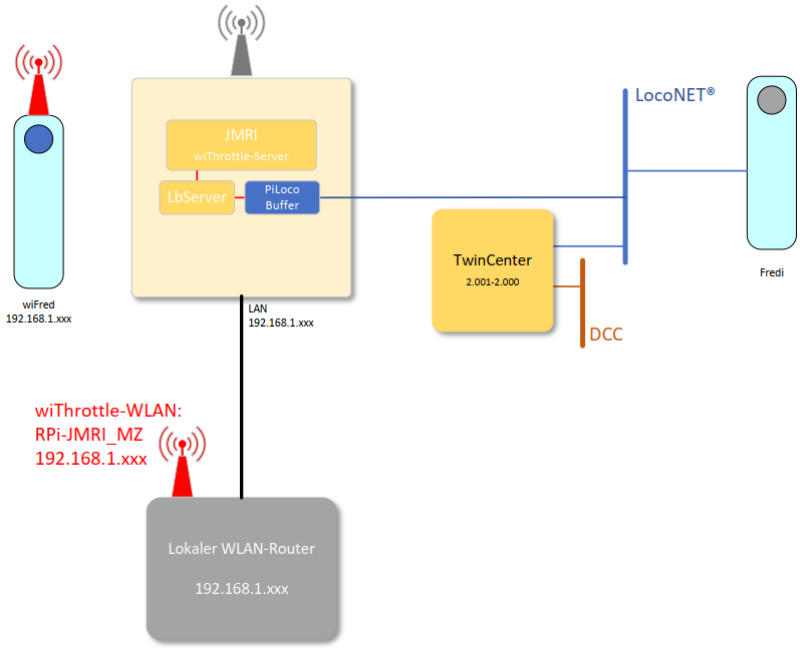
Versionsgeschichte dieses Dokuments....2
PiLocoBuffer – was ist das?....3
Eines vorweg…....3
Wo finde ich was?....4
PiLocoBuffer – System erstellen....5
PiLocoBuffer – die Platine....5
Der Raspberry Pi....5
Der externe Router....5
Die Konfiguration des externen Routers....5
PiLocoBuffer als WiThrottle Server konfigurieren....6
Was wird benötigt....6
SD-Karte mit Raspberry Pi Imager vorbereiten....6
Jetzt ist der Raspberry an der Reihe - einschalten....8
Jetzt ist der Raspberry an der Reihe – zusätzlich benötigte Software installieren....9
1. LBServer installieren....9
2. JAVA und JMRI installieren....9
3. JMRI konfigurieren....10
Sprache einstellen (optional)....11
Verbindung zum LbServer einrichten....11
wiThrottle-Server aktivieren....12
wiThrottle-Server und die Funktion F2 am wiFRED....12
4. Startverzögerung für JMRI....13
Was man noch Anpassen kann....13
wiThrottle-Server – eine Alternative zu JMRI – die neue Lösung von Pål Olsen....14
Startbefehl auf dem Raspi....14
Betrieb mit der ReadyToGo-Version des PiLocoBuffer....14
1. Pfad zur .jar-Datei und Java testen....14
2. Crontab bearbeiten....15
3. Autostart-Befehl einfügen....15
4. Speichern und beenden....15
5. Raspberry Pi neu starten....15
PiLocoBuffer – Handhabung / Bedienung....16
Bedeutung der LED....16
Ausschalten des Raspberry....16
PiLocoBuffer – Gehäuse....16
SD-Karte kopieren? Ganz so einfach geht es nicht…....16
Anhang A: Kontrolle des wiThrottle-Servers mit einem Monitor....17

# Versionsgeschichte dieses Dokuments

| | |
|---|---|
| 26.08.2024 | Initiale Erstellung |
| 21.09.2024 | Links korrigiert |
| 02.10.2024 | redaktionelle Korrekturen |
| 14.01.2025 | Hinweis zu meinen verwendeten WLAN-Handreglern hinzugefügt |
| 11.03.2024 | Abschnitt *SD-Karte kopieren? Ganz so einfach geht es nicht…* hinzugefügt |
| 04.06.2025 | Abschnitt „wiThrottle-Server und F2“ hinzugefügt |
| 17.06.2025 | Abschnitt „wiThrottle-Server – eine Alternative zu JMRI“ hinzugefügt |
| 05.01.2026 | redaktionelle Korrekturen zum Thema „wiThrottle-Server“ |
| 21.08.2026 | redaktionelle Korrekturen |

# PiLocoBuffer – was ist das?

Bereits in Krümelbahn Info 3 - Raspberry als wiThrottle-Server (https://github.com/Kruemelbahn/Infoletter/blob/main/Modellbahn/Handregler/Krümelbahn%20Info%203%20-%20RaspBerry%20als%20wiThrottle-Server.pdf) habe ich beschrieben, wie ein Raspberry Pi als Schnittstelle zwischen WLAN-Handreglern (auf Basis des wiThrottle-Protokolls) und einer DCC-Zentrale zum Einsatz kommt.
Im FREMO ist man nun einen Schritt weiter gegangen und hat für den Raspberry eine Platine entwickelt, auf der eine Schnittstellensoftware seine Arbeit verrichtet. Diese Platine ist als HAT[1] entwickelt worden und wird auf den Raspberry aufgesetzt. Auf dem Raspberry selbst läuft JMRI und der LbServer (https://loconetovertcp.sourceforge.net/Server/index.html#lbserver). Das Ganze nennt sich dann ***PiLocoBuffer*** und kommt mit minimaler Hardware und Verkabelung aus:




## Eines vorweg…

Im FREMO gab (und gibt es auch immer wieder in unregelmäßigen Abständen) eine Sammelbestellung, in der der PiLocoBuffer auch als ReadyToGo-Variante angeboten wurde. Für die Besitzer dieser Variante ist diese Anleitung fast nicht von Relevanz – es sei denn:

- man hat Probleme mit der Funktion F2 am wiFRED – dafür gibt es dann den Abschnitt wiThrottle-Server und die Funktion F2 am wiFRED, in dem eine mögliche Lösung aufgezeigt wird
- oder möchte etwas über PiLocoBuffer – Handhabung / Bedienung wissen

[1] HAT = **H**ardware **A**ttached on **T**op, eine Platine als Aufsatz für einen Raspberry, ähnlich einem Shield für den Arduino

## Wo finde ich was?

| | |
|---|---|
| PiLocoBuffer - Hardware | Entweder:<br>- Direktbezug[2] mit Platine und Bauteilen über den FREMO<br>- oder mit allen Informationen und Fertigungsdaten hier:<br>https://loconetovertcp.sourceforge.net/Interface/PiLocoBuffer/index.html |
| Zusammenbau des PiLocoBuffer | https://loconetovertcp.sourceforge.net/Interface/PiLocoBuffer/how-to-build.html |
| FREMO-Lösung zur Anbindung von WIFI-Handreglern und mehr… | https://www.modellbahn-online.de/1887-2/<br>https://www.fremo-net.eu/fileadmin/hp1_archiv/Hp1_24_3_col_web.pdf |
| Gehäuse für den PiLocoBuffer | https://makerworld.com/en/models/204000<br>https://sourceforge.net/projects/loconetovertcp/files/PiLocoBuffer/case/ |
| Krümelbahn Info 3 - Raspberry als wiThrottle-Server | https://github.com/Kruemelbahn/Infoletter/blob/main/Modellbahn-/Krümelbahn%20Info%203%20-%20RaspBerry%20als%20wiThrottle-Server.pdf |
| Krümelbahn Info 11 - JMRI - Universalwerkzeug für die Modellbahn | https://github.com/Kruemelbahn/Infoletter/blob/main/Modellbahn-/Krümelbahn%20Info%2011%20-%20JMRI%20-%20Universalwerkzeug%20für%20die%20Modellbahn.pdf |
| wiThrottle-Protokoll | https://www.jmri.org/help/en/package/jmri/jmrit/withrottle/Protocol.shtml |
| Von mir verwendete WLAN-Handregler | wiFred (H.Rosemann)<br>- https://www.wifred.de/<br>- https://github.com/newHeiko/wiFred<br>wiThrottle (A.Heckt)<br>- https://gitlab.com/fsmd/electronics/withrottle<br>Android-App ‚EngineDriver'<br>- https://enginedriver.mstevetodd.com/ |
| alternativer wiThrottleServer | https://www.modellbahn-online.de/alternative-zum-withrottleserver/ |

[2] Für FREMO-Mitglieder; Kosten der Platine mit allen Bauteilen – wenn verfügbar - ca. 22€ zzgl. Versandkosten [Stand: 26.8.2024]

# PiLocoBuffer – System erstellen

## PiLocoBuffer – die Platine

Bezieht man die Platine über den FREMO, so sind alle SMD-Bauteile bereits bestückt, es müssen nur noch Stecker, LCD, LEDs und Taster montiert werden. Hierzu sind keine großartigen Lötkenntnisse erforderlich, da es sich nicht um SMD-Bauteile handelt.
Dieser Zusammenbau ist hier https://loconetovertcp.sourceforge.net/Interface/PiLocoBuffer/how-to-build.html (in Englisch) beschrieben.

## Der Raspberry Pi

Zum Einsatz kommt bei mir:

```
Pi4 Model B, 4GB, 2*USB2.0, 2*USB3.0
SD-Card: 32GB (min: 16GB, max:64GB)
PiLocoBuffer-Platine
```

## Der externe Router

Der Raspberry Pi selbst hat bereits einen WLAN-Chip on Board. Dieser arbeitet in seiner Grundkonfiguration als Slave, WLAN-Geräte sehen zwar das Netzwerk, können sich aber nicht mit ihm verbinden. Das Image-File von Steve Todd (https://mstevetodd.com/jmri-raspberrypi-access-point) verwandelt den Raspberry in einen Access-Point und behebt so diesen Mangel, nachteilig bleibt jedoch die geringe Reichweite des WLAN und die maximale Anzahl von acht Geräten, die sich mit dem Raspberry verbinden können.

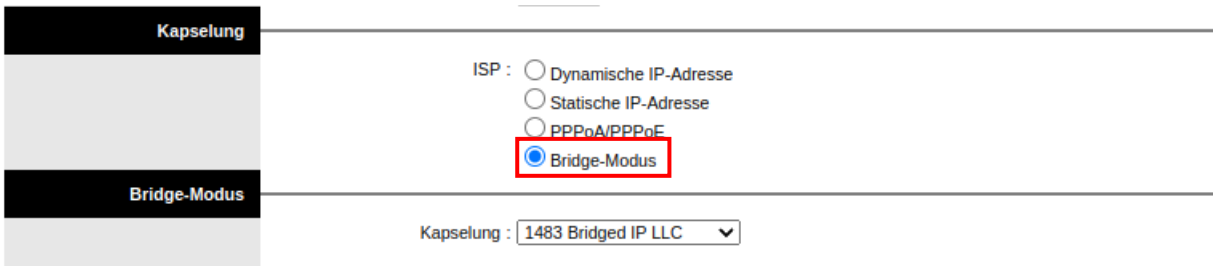
Um zum einen nicht den Raspberry in einen Access-Point aufrüsten zu müssen, zum anderen um bei Modultreffen besser aufgestellt zu sein, kommt auch bei mir ein externer WLAN-Router zum Einsatz. Als Router für das WLAN gibt es viele nutzbare Geräte. Im FREMO beliebt ist eine (auch alte) FritzBox. Den von mir verwendeten Router

```
TP-LINK-Router TD-W8961NB
```

hatte ich noch im Bestand. Das integrierte ADSL-Modem für einen Internetzugang wird nicht benötigt und daher stillgelegt (die Zugangsdaten gelöscht) und nicht ans Internet angeschlossen.

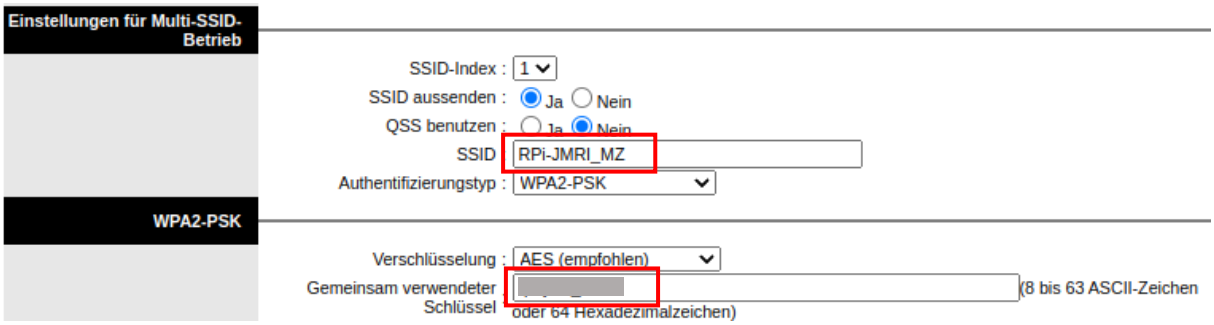
## Die Konfiguration des externen Routers

Der ADSL-Modus (PPPoA/PPPoE) wird abgeschaltet[3] und der Bridge-Modus eingeschaltet:




Die Zugangsdaten für das WLAN (neu) einstellen – entsprechend den Raspberry-Einstellungen:

[3] Die Einstellungen sind bei jedem Router anders: hier hilft das Handbuch des Routers…




## PiLocoBuffer als WiThrottle Server konfigurieren

… unter diesem Namen gibt es im Internet eine ausführliche Beschreibung von M. Hellmann (https://web.archive.org/web/20250418025123/https://us-trains.de/?p=390, hier fehlen leider einige Bilder, aber die kommen ja auch im weiteren Text 😊 ).

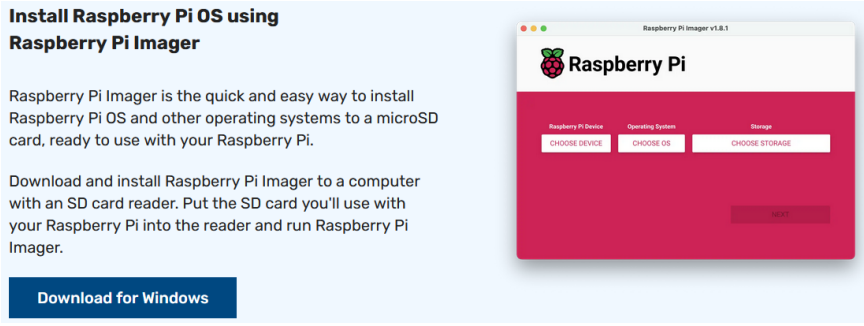
Auch wenn es jetzt wie ein Plagiat aussieht – für den Lesefluss und den Installationsfortschritt werden hier weite Teile zitiert (…und um einige kleine Fehler korrigiert 😊 ):

### Was wird benötigt

- Raspberry Pi 4 Model B mit 2 GB RAM (***keinen Pi 5***[4]), 4 GB geht auch, braucht es aber nicht
- PiLocoBuffer (aus der FREMO-Bestellaktion)
- Schnelle Micro-SD-Karte 32 GB, ggf. Adapter auf SD-Karte – abhängig vom Kartenslot des Windows PC
- Leistungsfähige Stromversorgung für den Pi (5V, 3A z.B. original Pi-Netzteil)
- Adapter oder Adapterkabel Micro-HDMI auf HDMI oder was euer Monitor hat (wird nur zur Einrichtung benötigt)

### SD-Karte mit Raspberry Pi Imager vorbereiten

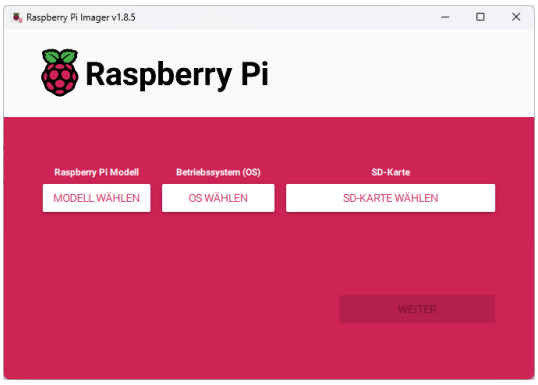
Das OS[5] für den Raspberry wir mittels des Raspberry Pi Imager auf die SD-Karte kopiert. Der Imager wird daher zunächst von https://www.raspberrypi.com/software/ heruntergeladen:




Über Download for Windows den Imager herunterladen und speichern.

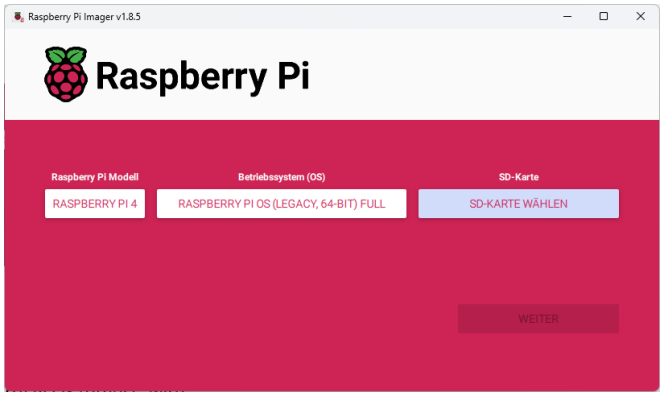
[4] Für einen Pi 5 (OS *Bookworm* oder *Trixie*) wird ein neuer LbServer benötigt, der diese OS unterstützt…
[5] OS = Operating System = Betriebssystem

Anschließend wird der Imager installiert und aufgerufen:

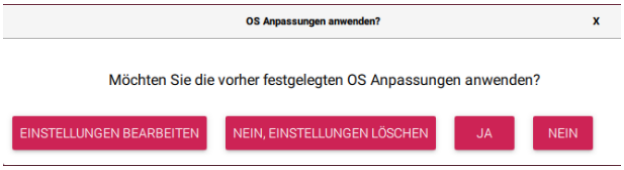



Unter

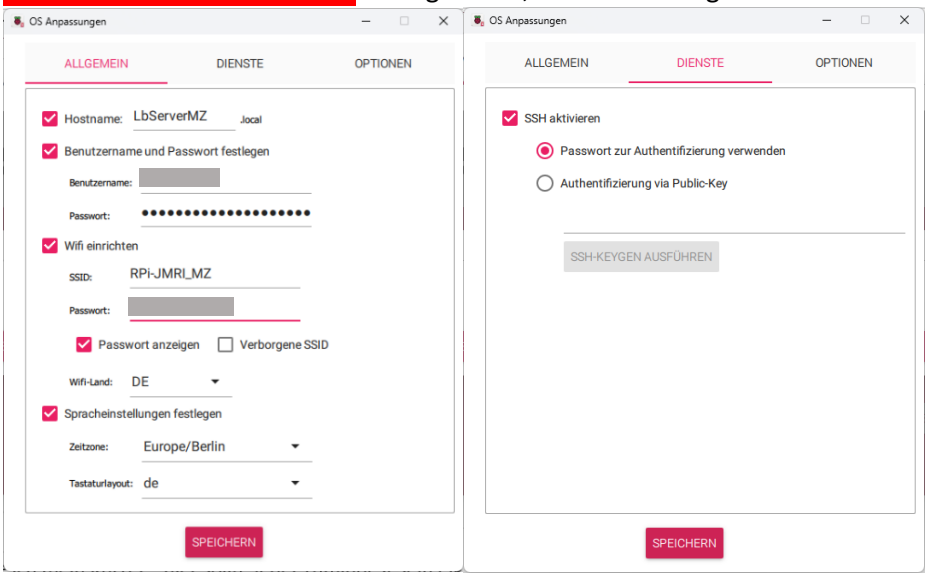
- Modell wählen wird der zu verwendende Raspberry ausgewählt, bei mir **Raspberry Pi 4**
- OS WÄHLEN wird das Betriebssystem ausgewählt:
  - Über **Raspberry Pi OS (other)** wird anschließend
  - **Raspberry Pi OS(Legacy, 64-Bit) Full** ausgewählt




- Fehlt nur noch SD-KARTE WÄHLEN - also machen wir das auch…
- Mit WEITER geht es zur nächsten Frage:



EINSTELLUNGEN BEARBEITEN ermöglicht es, zusätzliche Angaben zu machen:




Auf der Seite ALLGEMEIN verwende ich bei einigen Einstellungen mein Kürzel – hier sollte jeder zumindest sein eigenes Kürzel (oder gänzlich andere Angaben verwenden). Unterschiedliche Kürzel (oder andere Namen) haben den Vorteil, dass auch mehrere WLAN-Netze für wiFREDs auf Treffen eingesetzt werden können…

- Mit SPEICHERN werden die Änderungen übernommen
- Und im nachfolgenden Dialog




wird mit JA die SD-Karte erstellt.

Nach der abschließenden Verifizierung der SD-Karte (ggf. eine Nachfrage zur Formatierung ignorieren) wird die Karte aus dem PC entnommen und in den noch spannungslosen Raspberry gesteckt.

## Jetzt ist der Raspberry an der Reihe - einschalten

- Der Raspberry ist zunächst spannungslos!
- PiLocoBuffer-Platine – sofern noch nicht geschehen – auf den Raspberry aufsetzen
- Mit dem Imager bespielte SD-Karte einsetzen
- Nur für die Inbetriebnahme wird benötigt:
  - Bildschirm über den (linken) Micro-HDMI-Port anschließen[6]
  - Maus und Tastatur anschließen
  - Netzwerkkabel zum häuslichen Router (noch nicht der Router anschließen
  - VNC ist nicht aktiviert und steht hier nicht zur Verfügung…
- Raspberry mit Spannung versorgen – um Spannungsversorgungsproblemen aus dem Weg zu gehen nutzt man hier nur ein originales Raspberry-Netzteil!
- Der Raspberry sollte jetzt hochfahren und abschließend einen Sonnenuntergang auf dem Bildschirm anzeigen. Ein anderes Bild zeigt die Verwendung eines falschen OS – also noch mal mit dem Imager neu starten.
- Über die Weltkugel die Internetverbindung testen

[6] Es wird ein Kabel „Micro-HDMI zu HDMI“ oder „Micro-HDMI zu DVI“ benötigt

## Jetzt ist der Raspberry an der Reihe – zusätzlich benötigte Software installieren

Die Einrichtung / Softwareinstallation gliedert sich in vier Teile.

### 1. LBServer installieren

```
sudo apt install pkg-config avrdude libsystemd-dev
```

Die Frage `Do you want to continue? [Y/n]` mit `Y` bestätigen.

Dann die Konfigurationsdatei herunterladen[7]:

```
wget https://sourceforge.net/projects/loconetovertcp/files/↵
PiLocoBuffer/config/PiLocoBufferConfig_0.1.zip
```

Der Download liegt anschließend im Stammverzeichnis des aktuellen Users. Über den File Manager (rechtes Fenster) das zip markieren, rechte Maustaste und „Extract Here“. Es wird ein neues Verzeichnis `PiLocoBufferConfig_0.1` erstellt, in dem sich dann die Konfigurationsdateien vom PiLocoBuffer befinden.

Weiter als Administrator[8]:

```
cd PiLocoBufferConfig_0.1
sudo make FIRMWARE_VER=2023-04-15 LBSERVER_VER=0.14 all
```

Nach einem Neustart (geht auch über das Terminal mit dem Befehl `sudo reboot`) sollte nun etwas auf dem LCD des PiLocoBuffer erscheinen – wird es aber in 95% der Fälle nicht! Nun ist erstmal drehen im Uhrzeigersinn am Poti (10k) links neben dem roten Knopf angesagt – drehen, drehen und weiter drehen, … – es können 25, 50, … Mal sein bis da langsam etwas erscheint. Mit dem anderen Poti (rechts) kann dann noch die Helligkeit passend eingestellt werden.

### 2. JAVA und JMRI installieren

```
sudo apt install default-jdk
```

im Laufe der Installation muss man wieder mit `Y` bestätigen.

Das neuste JMRI bekommt am schnellsten über das Terminal:

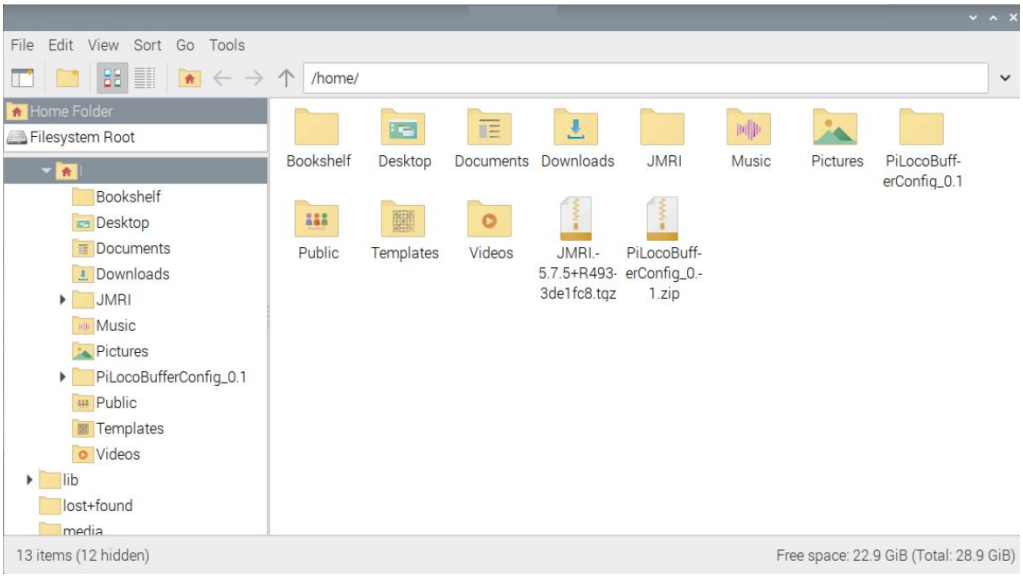
```
wget https://github.com/JMRI/JMRI/releases/download/v5.6/↵
JMRI.5.6+R89a87446cb.tgz
```

Der Download landet dann im Stammverzeichnis des aktuellen Users, dort mit dem File Manger die tgz-Datei markieren, rechte Maustaste und „Extract Here“. Es wird ein neues Verzeichnis `JMRI` erstellt. Dort liegen dann die Programmdateien von JMRI.

[7] ↵ ist im Folgenden das Zeichen für „Fortsetzung der Zeile“

[8] Die Verwendung von `sudo` ermöglicht die Befehlsausführung als Administrator

Es sollte dann so oder so ähnlich aussehen:




Weiter im Terminal:

```
cd ~/JMRI
./CreateJmriApps.sh -d
```

Nun sollte auf dem Desktop `PanelPro.desktop` erscheinen.

Da wir wollen, dass der PiLocoBuffer automatisch startet, wenn er eingeschalten wird, geht es wieder ins Terminal:

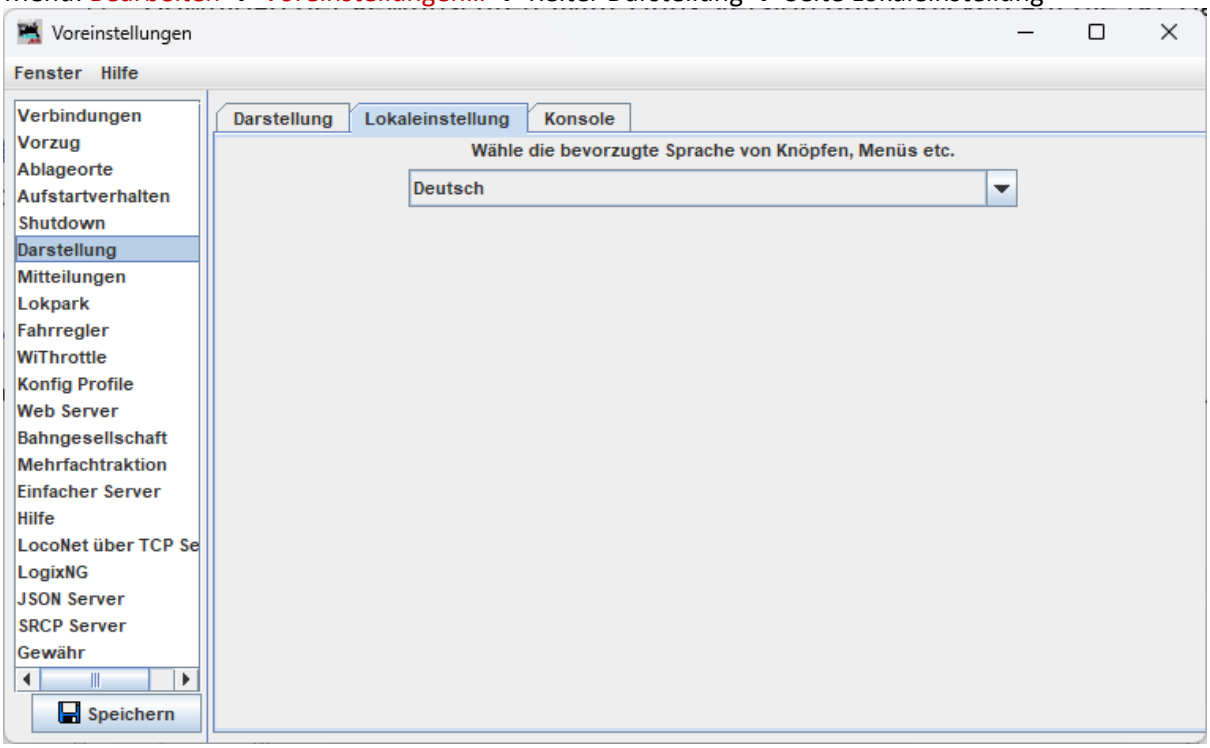
```
cd ~/.config
mkdir autostart
cd autostart
ln -s ~/Desktop/PanelPro.desktop PanelPro.desktop
sudo reboot
```

## 3. JMRI konfigurieren

Nach dem Hochfahren startet DecoderPro® nun und muss konfiguriert werden.

## Sprache einstellen (optional)

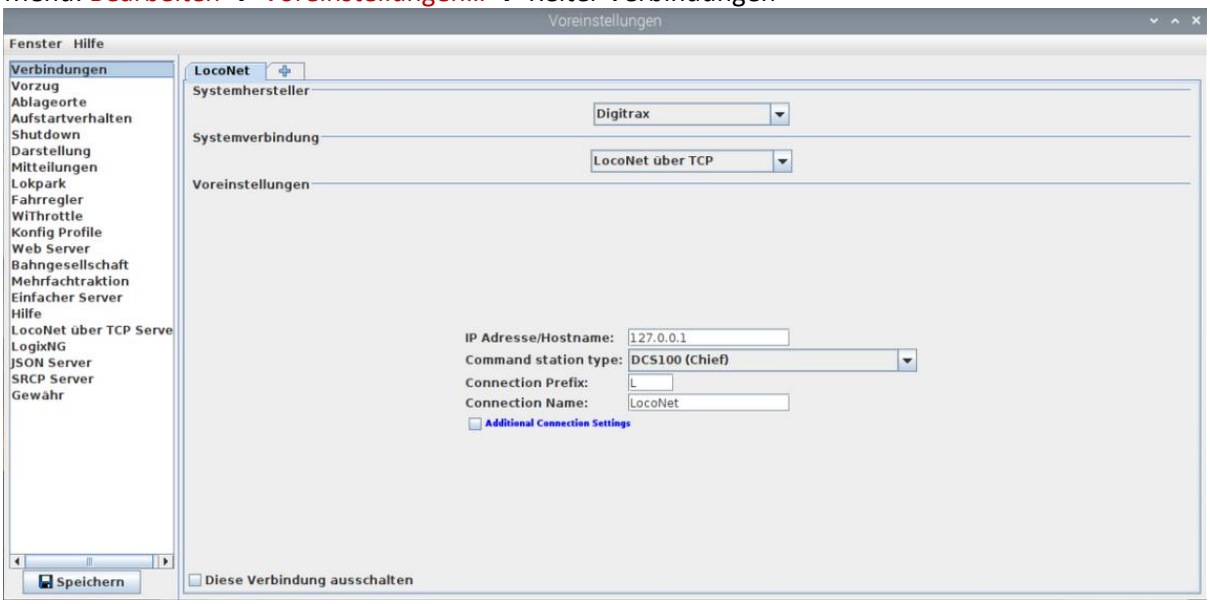
Menü: Bearbeiten → Voreinstellungen… → Reiter Darstellung → Seite Lokaleinstellung




*(optional, weil der Raspberry ohne Monitor zum Einsatz kommt…)*

## Verbindung zum LbServer einrichten

Menü: Bearbeiten → Voreinstellungen… → Reiter Verbindungen

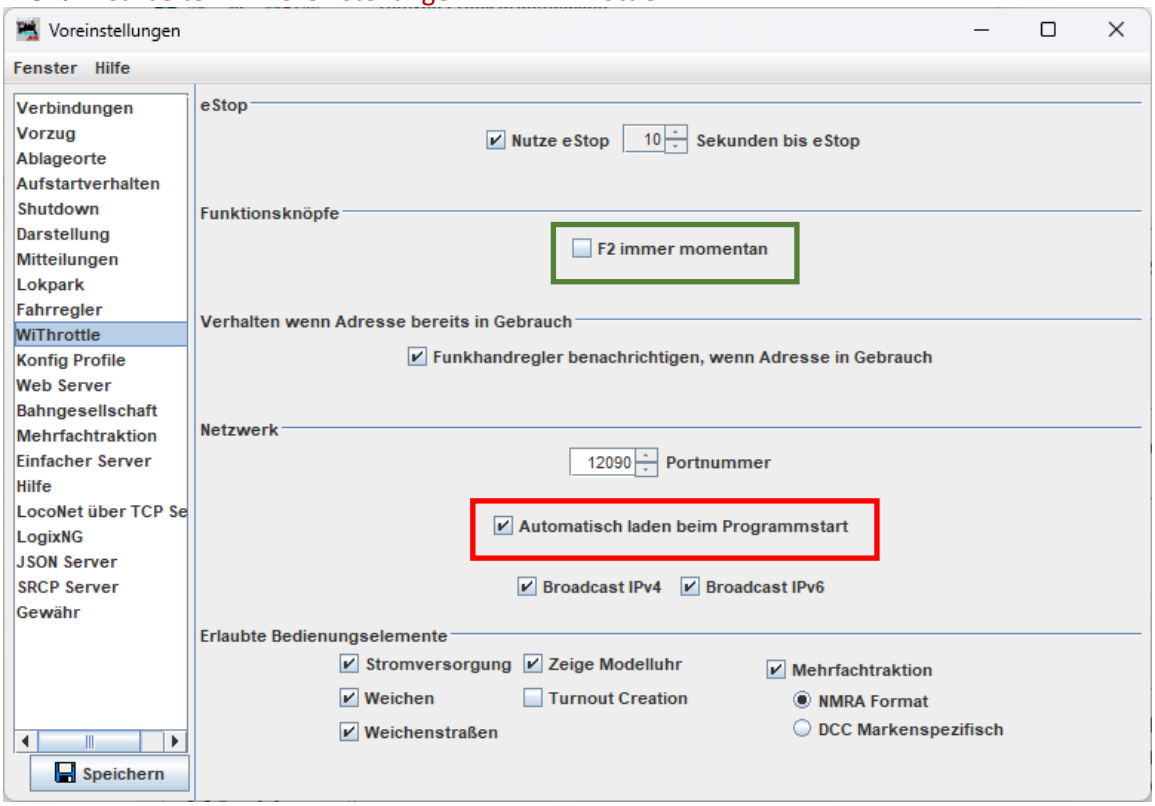



- Digitalsystem `Digitrax`
- Verbindung `Loconet über TCP`
- Localhost `127.0.0.1` als Serveradresse
- `DSC100` als Zentrale
- `LocoNet` als Verbindungsname

und speichern.

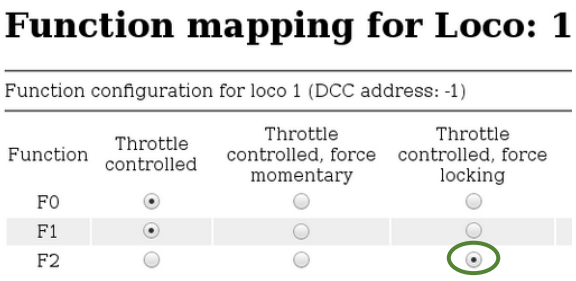
## wiThrottle-Server aktivieren

Menü: Bearbeiten → Voreinstellungen… → WiThrottle




## wiThrottle-Server und die Funktion F2 am wiFRED

Hinweis bei Nutzung des wiFRED: auch wenn – wie im obigen Dialog (grüner Rahmen) zu sehen – die Einstellung F2 immer momentan **nicht** aktiviert ist, wirkt die F2-Taste am wiFRED immer als Momentan-Funktion, d.h. sobald die Taste wieder losgelassen wird, wird die F2-Funktion wieder abgeschaltet. Abhilfe schafft hier die Einstellungen auf der wiFRED-Konfigurationsseite für die entsprechende Lok:




Diese Einstellung wirkt nur auf die Lok, für die sie aufgerufen wurde – steht also für jede Lok 1…4 auf dem wiFRED separat zur Verfügung.
Um also F2 wie eine ‚normale' Funktion zu verwenden, sind nachfolgende Maßnahmen erforderlich:

- In JMRI wird die Funktion F2 immer momentan abgeschaltet
- Im wiFRED wird für jedes Fahrzeug F2 Throttle controlled, force locking aktiviert

Experteninfo: JMRI ist ein amerikanisches Projekt und auf dem amerikanischen Markt ist an vielen Fahrzeugdecodern ein Signalhorn angeschlossen, welches mit F2 gesteuert wird. Und damit das Signalhorn nur aktiv ist, wenn die F2-Taste gedrückt ist, ist die Einstellung F2 immer momentan als Voreinstellung aktiviert.
Siehe auch hier: Github für JMRI

### 4. Startverzögerung für JMRI

Nun gibt es noch eine Besonderheit. Wenn man das ganze System (Router/Zentrale/PiLocoBuffer) gleichzeitig einschaltet (wenn auf einem Treffen die komplette Stromversorgung des Arrangements einsteckt wird), startet unter Umständen der Raspberry mit JMRI schneller, als der Router das WLAN bereitstellen kann: d.h. JMRI findet beim Start keine LAN/WLAN-Verbindung. Somit können sich die wiFREDs nicht anmelden.

Daher macht es Sinn den Start von JMRI etwas zu verzögern[9].

Durch den ersten Start von JMRI wird im Homeverzeichnis der Ordner `.jmri` automatisch erstellt. Der Ordner ist aber „Hidden“ und wird erst angezeigt, wenn man im Menü „View“ den Menüpunkt „show Hidden“ aktiviert. In dem Ordner befindet sich die Datei `PanelProConfig2.properties`. Diese Datei wird nun editiert:

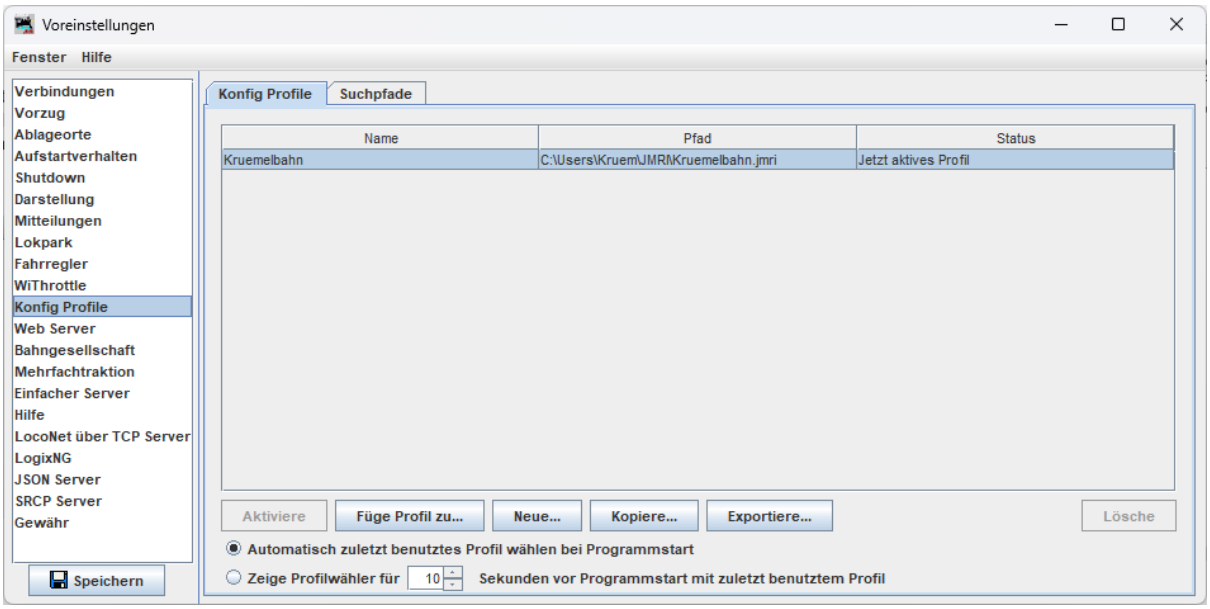
```
<entry key="autoStartTimeout">30</entry>
<entry key="autoStart">false</entry>
```

JMRI muss nun neu gestartet werden und damit ist der PiLocoBuffer einsatzbereit.

### Was man noch Anpassen kann

Beim Starten wartet JMRI immer auf die Auswahl des Konfigurationsprojekts. Wenn es nur ein Konfigurationsprojekt gibt (was ja der Standardfall sein wird), kann man den Dialog umgehen:

Menü: Bearbeiten → Voreinstellungen… → Konfig Profile




Zum einen können hier nicht benötigte Profile gelöscht werden, zum anderen kann hier „Automatisch zuletzt benutztes Profil wählen bei Programmstart“ aktiviert werden.

Da hierdurch die Startzeit verkürzt wird, ist ggf. der Wert für die Startverzögerung für JMRI anzupassen.

[9] Der Zeitwert (im Beispiel `30` Sekunden) kann je nach eingesetztem Router deutlich kürzer oder länger sein.

# wiThrottle-Server – eine Alternative zu JMRI – die neue Lösung von Pål Olsen

Im FREMO wurde eine Alternative zum JMRI-wiThrottle-Server entwickelt:

- WiThrottle-Server von Pål Olsen

Dieser neue wiThrottle-Server

- benötigt zwar weiterhin Java, macht aber auch JMRI an dieser Stelle überflüssig.
- umgeht auch die F2-Problematik

Die nachfolgende Beschreibung wurde von R. Bäcker erstellt (https://www.modellbahn-online.de/alternative-zum-withrottleserver/)

Auch wenn es jetzt wie ein Plagiat aussieht – für den Lesefluss und den Installationsfortschritt werden hier weite Teile zitiert:

Wer eine ressourcenschonende Lösung ohne grafische Oberfläche sucht, findet mit der Lösung von Pål Olsen (ich nenne die hier mal PWTS) eine funktionale Alternative, z. B. zu JMRI.

Die Anwendung ist in Java geschrieben und lässt sich problemlos auf einem **Raspberry Pi** betreiben, z. B. in Kombination mit einem **PiLocoBuffer**.

Diese Kombination eignet sich besonders für den Betrieb mit wiFRED´s.

Da PWTS keine eigene LocoNet-Schnittstelle besitzt, erfolgt die Verbindung über einen vorhandenen **LnOverTcpServer** wie etwa lbserver.

Der PWTS kann hier heruntergeladen werden:
http://lysthuset.homeip.net/WiThrottleServer.jar

## Startbefehl auf dem Raspi

```
java jar WiThrottleServer.jar [lbserver host] [lbserver port]
```

## Betrieb mit der ReadyToGo-Version des PiLocoBuffer

Anleitung zum Testen mit der ReadyToGo-Version und Einrichten Autostart:

Voraussetzung:

- die heruntergeladene Datei `WiThrottleServer.jar` liegt unter `/home/pi/WiThrottleServer/`
- **Der Autostart von JRMI muss unterbunden werden.**
  **Dazu die beiden Dateien im Ordner `/home/pi/Desktop` umbenennen (z. B.: .alt anhängen).**

### 1. Pfad zur .jar-Datei und Java testen

Manuell testen im Terminal:

```
/usr/bin/java -jar /home/pi/WiThrottleServer/WiThrottleServer.jar ↵
localhost 1234
```

Wenn die Ausgabe lautet:

**Server is STARTING…**

**Server is now RUNNING**

funktioniert alles korrekt.

## 2. Crontab bearbeiten

Crontab-Editor für den Benutzer pi öffnen:

```
crontab -e
```

Beim ersten Aufruf den Editor `nano` wählen, wenn danach gefragt wird.

## 3. Autostart-Befehl einfügen

Ganz unten einfügen:

```
@reboot sleep 15 && /usr/bin/java -jar ↵
/home/pi/WiThrottleServer/WiThrottleServer.jar localhost 1234 > ↵
/home/pi/withrottle.log 2>&1↵
```

Erklärung:

- `sleep 15` = wartet 15 Sekunden nach dem Boot (Netzwerkzeit)
- `/home/pi/withrottle.log 2>&1` = schreibt alle Ausgaben (auch Fehler) in eine Logdatei

### *4. Speichern und beenden*

In **nano**:

- CTRL + O → Speichern
- ENTER → bestätigen
- CTRL + X → beenden

## 5. Raspberry Pi neu starten

```
sudo reboot
```

**Verbundene wiFRED´s sollten sich nach 1-2 Minuten verbinden, sonst ist die Funktion zu prüfen.**

```
ps aux | grep WiThrottle
cat /home/pi/withrottle.log
```

Wenn in der Logdatei steht:

**Server is STARTING…**

**Server is now RUNNING**

ist der Autostart erfolgreich eingerichtet!

## PiLocoBuffer – Handhabung / Bedienung

Die Handhabung bzw. Bedienung des PiLocoBuffer ist einfach:

- LocoNET® anschließen, die Zentrale sollte bereits aktiv sein
- Router über Netzwerkkabel mit Raspberry verbinden
- Router Spannungsversorgung anschließen
- Raspberry Spannungsversorgung anschließen
- Und zum Schluss Router und Raspberry gemeinsam einschalten

Fertig 😊

Nach dem Einschalten erscheint beim Hochfahren des PiLocoBuffer:




und wenn der PiLocoBuffer betriebsbereit ist:




Für den Standardbetrieb werden am PiLocoBuffer weder Bildschirm noch Tastatur und Maus benötigt.

### Bedeutung der LED

- *rot* leuchtet beim Starten des Raspberry kurz auf, weiter nicht in Gebrauch
- *gelb* wird kurzzeitig für jede an das LocoNET® gesendete Meldung eingeschaltet
- *grün* wird kurzzeitig für jede vom LocoNET® empfangene Meldung eingeschaltet

### Ausschalten des Raspberry

Die **rote** Taste wird verwendet, um das Herunterfahren des Raspberry auszulösen:




Es wird empfohlen, diese Taste tatsächlich zu verwenden, da bekannt ist, dass die SD-Karte durch das Ausschalten im Vollbetrieb beschädigt werden kann - vor allem, wenn sie im beschreibbaren Modus verwendet wird, was die Standardeinstellung ist.

## PiLocoBuffer – Gehäuse

Inzwischen wurden durch Mitglieder des FREMO auch Gehäuse für den Raspberry Pi mit Locobuffer-Platine und LCD entworfen. 3D-Druckdateien können frei heruntergeladen werden:

https://makerworld.com/en/models/204000

https://sourceforge.net/projects/loconetovertcp/files/PiLocoBuffer/case/

## SD-Karte kopieren? Ganz so einfach geht es nicht…

Wer jetzt glaubt, es genügt, eine vorhandene – von einem anderen PiLocoBuffer kopierte – SD-Karte einfach in ein neues System einzusetzen, dem sei gesagt: ganz so einfach ist es nicht.

Die SD-Karte enthält zwar die auf dem Raspberry benötigte Software – der AVR auf der PiLocoBuffer-Platine kennt seine Aufgabe (noch) nicht – der Installationsschritt 1. LBServer installieren ist daher zwingend erforderlich.

## Anhang A: Kontrolle des wiThrottle-Servers mit einem Monitor

Startbildschirm von JMRI:




Ansicht des wiThrottle-Servers:

